



预备知识

Python和Jupyter Notebook

包管理器



包 (Package)

- 在Python中，包 (Package) 本质上是一个包含多个模块的目录，它让我们能够组织和管理相关的代码文件。
- 包可以用Python语言编写，也可以用其他语言编写。Python的很多核心库和性能关键型库都是用C/C++编写的（这种方式称为扩展模块）。
- 使用包的优点：
 - 代码复用：不用重复造轮子
 - 模块化：代码组织更清晰
 - 命名空间隔离：避免命名冲突
 - 版本控制：可以管理不同版本的依赖

包管理器

- 包管理是指对软件包（包括安装、更新、配置和卸载）进行自动化的过程。
- 源（Source）：在包管理中通常指的是包索引服务器或软件仓库，可以理解为Python包的"应用商店"。
 - 最著名的官方源是 PyPI（Python Package Index）。
- 由于网络原因，直接访问PyPI可能较慢。国内常用的镜像源有：

```
# 阿里云镜像
https://mirrors.aliyun.com/pypi/simple/

# 清华大学镜像
https://pypi.tuna.tsinghua.edu.cn/simple/

# 中国科技大学镜像
https://pypi.mirrors.ustc.edu.cn/simple/

# 豆瓣镜像
https://pypi.douban.com/simple/
```

- pip是Python官方推荐的包管理工具，从Python 3.4开始就内置在Python安装包中。
- 使用pip包管理器安装包：

```
# 基本安装
pip install requests

# 安装指定版本
pip install requests==2.28.1

# 安装版本范围
pip install "requests>=2.0,<3.0"

# 从requirements文件安装
pip install -r requirements.txt

# 安装到用户目录（无需管理员权限）
pip install --user requests

# 使用指定镜像源安装
pip install -i https://pypi.tuna.tsinghua.edu.cn/simple requests
```

- 卸载包:

```
pip uninstall requests
```

- 查看已经安装的包:

```
# 列出所有已安装包  
pip list  
  
# 以表格形式列出  
pip list --format=table  
  
# 查看过时的包  
pip list --outdated  
  
# 查看特定包的信息  
pip show requests
```

- 升级包:

```
# 升级pip本身  
pip install --upgrade pip  
  
# 升级特定包  
pip install --upgrade requests
```

写Pythonic的代码



写Pythonic的代码

- Pythonic是一个形容代码风格的术语，指的是符合Python语言设计哲学和惯用法的代码。
- 简单来说，就是"用Python的方式写Python代码"。
- Pythonic不是强制规范，而是Python社区积累的最佳实践。
- 为什么追求Pythonic?
 - 可读性强：其他Python开发者能轻松理解
 - 效率高：通常利用了Python的优化特性
 - 符合习惯：遵循社区共识，减少沟通成本
 - 更少的Bug：简洁的代码往往更少出错

- 多重赋值

```
# 非Pythonic  
a = 1  
b = 2  
c = 3  
  
# Pythonic  
a, b, c = 1, 2, 3
```

- 链式比较

```
# 非Pythonic  
if x > 5 and x < 10:  
  
# Pythonic  
if 5 < x < 10:
```

- 三元表达式

```
# 非Pythonic
if age >= 18:
    status = "成年"
else:
    status = "未成年"

# Pythonic
status = "成年" if age >= 18 else "未成年"
```

- 列表推导

```
# 非Pythonic
squares = []
for i in range(10):
    if i % 2 == 0:
        squares.append(i**2)

# Pythonic
squares = [i**2 for i in range(10) if i % 2 == 0]
```

数据结构的使用

• 使用Join连接字符串

```
# 非Pythonic
result = ""
for s in string_list:
    result += s + ","
result = result[:-1]

# Pythonic
result = ", ".join(string_list)
```

• 格式化字符串

```
name, age = "Alice", 25

# 非Pythonic
print("我叫" + name + ", 今年" +
      str(age) + "岁")

# 普通格式化字符串
print("我叫%s, 今年%d岁" % (name,
                             age))

# Pythonic f-string
print(f"我叫{name}, 今年{age}岁")
```

条件与循环

- 循环中的else

```
# 查找元素
for item in items:
    if condition(item):
        break
else:
    # 循环正常结束（没有break）时执行
    print("未找到符合条件的元素")
```

- zip并行迭代

```
names = ["Alice", "Bob", "Charlie"]
ages = [25, 30, 35]

# 非Pythonic
for i in range(len(names)):
    print(names[i], ages[i])

# Pythonic
for name, age in zip(names, ages):
    print(name, age)
```

Jupyter Notebook



Jupyter Notebook简介

- Jupyter Notebook是一个交互式开发环境，与传统代码编辑器的区别在于，它不仅
仅是写代码的工具，更是一种代码、结果和文档融为一体的工作方式



- Jupyter这个名字是由三种编程语言（Julia、Python、R）的名称组合而成

- Jupyter Notebook可以编辑专门的.ipynb 文件，在一个.ipynb文件中，你可以同时写：

- 代码（Code Cells）：支持多种编程语言，最常用的是Python，也可以运行R、Julia等
- 运行结果（Output Cells）：比如数据表格、绘制的图表、计算结果、甚至交互式控件
- 文字说明（Markdown Cells）：可以写分析过程、结论、思路，支持 Markdown 语法和 LaTeX 数学公式，还能插入图片

为什么要用 Jupyter Notebook

- 和传统的软件开发不同，机器学习算法开发需要进行大量数据处理分析工作，而数据分析有很强的探索性，Notebook的设计了这种**探索 - 验证 - 总结**的工作流。
 - 不是一开始就能写出完整的程序，而是需要边看数据、边尝试、边调整。
 - Notebook文件支持代码、Markdown文本的穿插，既是代码，又是实验报告。实际使用中，Notebook是一个可运行的研究报告，既能做实验，又能直接分享成果。
 - Notebook目前已经成为数据科学、机器学习领域的事实标准。

安装和使用

● 安装 Jupyter Notebook

- 方式一：使用 Anaconda，它集成了Jupyter Notebook等常用工具
- 方式二：普通 Python 环境可通过 pip 安装：

```
pip install notebook
```

● 启动 Notebook

- 在终端或 Anaconda Prompt中进入你的项目目录，然后输入：

```
jupyter notebook
```

- 在命令行加 --notebook-dir=路径 可以指定启动目录，例如：

```
jupyter notebook --notebook-dir=D:/Projects
```

实验报告提交



实验报告提交

- 报告提交系统: <http://124.222.192.49:8080/>
- 注册

欢迎登录

登录

还没有账号 [立即注册](#)

务必保证学号、姓名输入正确!

注册账号

* 注册后需等待教师审核通过方可登录

注册

已有账号? [立即登录](#)

● 查看当前实验

📖 实验报告提交

☰ 实验列表

📄 我的提交

张三

实验列表

进行中的实验

实验1

>

PyTorch基本操作及前馈神经网络

📅 开始: 2026/4/8 10:30:00 ⌚ 截止: 2026/4/19 23:59:00

归档实验

暂无归档实验

注意实验提交的开始、结束时间，过期无法再提交！

实验报告提交

• 上传Jupyter Notebook

实验报告提交

实验列表我的提交张三

实验1

PyTorch基本操作及前馈神经网络

开始: 2026/4/8 10:30:00 截止: 2026/4/19 23:59:00

上传报告

上传 Jupyter Notebook (.ipynb)

格式: 学号+姓名+实验名称.ipynb (分隔符支持 +, -, _)

新的提交将覆盖旧的提交

提交记录

学号	姓名	文件名	状态	提交时间	操作
2023000000	张三	2023000000-张三-实验1.ipynb	提交成功	2026/4/8 17:25:56	PDF

上传后约需要处理10秒左右，处理完成自动生成报告PDF

实验报告提交

- 每次实验提供完整可运行的初始代码（Jupyter Notebook形式，可通过课程主页或QQ群下载）；
- 实验完成时，可自由修改现有代码，或加入新的内容，鼓励加入自己的思考和总结；
- 实验完成情况占总成绩的20%，取每次实验的平均成绩；
- 每次实验务必在截止日期前提交，否则当次实验计为0分。

Q&A